

Aprendizaje Profundo - Exercise 7 XGI

David Hartl, Vincenzo Manzo

January 12, 2026

Contents

1	Project Summary	2
2	Description of Framework and Architecture	3
2.1	PyTorch	3
2.2	Network Architecture: The MSClassifier	3
2.3	Interpretability Methodology (XAI)	3
2.3.1	Integrated Gradients (IG)	3
2.3.2	Gradient-weighted Class Activation Mapping (Grad-CAM)	3
2.3.3	Local Interpretable Model-agnostic Explanations (LIME)	4
3	Framework Installation and Configuration	5
3.1	Execution Environment	5
3.2	Library Installation	5
3.3	Environment Configuration and Reproducibility	5
3.4	Google Drive Integration	6
3.5	Dataset Structure Verification	6
3.6	DataLoader Preparation	6
3.7	Validation Split	6
3.8	Data Leakage Verification	6
3.9	Hardware Configuration	6
4	Training	7
4.1	Data Refinement and Preprocessing	7
4.2	Model configuration and training strategy	7
4.3	Learning rate scheduler	8
4.4	Loss function behavior	8
4.5	Overfitting prevention	8
5	Example of Use (Inference & XAI)	9
5.1	Inference Methodology	9
5.2	Interpretability Methodology	9
5.2.1	Gradient-weighted Class Activation Mapping (Grad-CAM)	9
5.2.2	Integrated Gradients (IG)	10
5.2.3	Local Interpretable Model-agnostic Explanations (LIME)	10
5.3	Visualization and Clinical Interpretation	10
5.3.1	Grad-CAM visualization	10
5.3.2	Integrated Gradients visualization	10
5.3.3	LIME visualization	10
5.3.4	Quantitative validation against lesion masks	11
6	Evaluation and Conclusion	12
6.1	Technical aspects: performance and reliability	12
6.2	Technical aspects: interpretability and coherence of explanations	12
6.3	Concluding remarks	14

1 Project Summary

This project investigates the applicability of Explainable Artificial Intelligence techniques within the domain of medical diagnostics, specifically targeting Multiple Sclerosis identification, and with this the recognition of lesions in MRI images. While Deep Learning models have achieved state-of-the-art performance in medical image analysis, their "black-box" nature remains a barrier to clinical adoption. The primary objective of this exercise is not to maximize classification metrics, but to show the internal reasoning processes of Convolutional Neural Networks when analyzing the MRI data.

The study utilizes the MSLesSeg data set [1], employing 2D axial slices in the FLAIR sequence to train a custom CNN architecture. Following the training phase, three distinct interpretability algorithms are implemented to generate saliency maps and local expressions.

1. **Integrated Gradients:** To axiomatically attribute prediction scores to input features.
2. **Grad-CAM (Gradient-weighted Class Activation Mapping):** To visualize coarse regions of interest based on the final convolutional layer activations.
3. **LIME (Local Interpretable Model-agnostic Explanations):** To provide local fidelity through perturbation-based approximations.

The analysis focuses on a qualitative evaluation of these explanations verifying whether the models focus areas correlate with pathologically validated lesions or if the network relies on spurious artifacts. This work aims to bridge the gap between high-performance neural computing and the interpretability required for reliable medical decision support systems.

Key Components:

- Dataset: MSLeg (FLAIR Sequencem Axial Plane) [1]
- Framework: PyTorch (with Captum and LIME libraries)
- Methodology: Supervised learning followed by post-hoc interpretability analysis.

2 Description of Framework and Architecture

2.1 PyTorch

PyTorch is an open-source deep learning framework originally developed by Meta AI [2]. It provides a flexible and efficient platform for building and training neural networks using dynamic computational graphs (eager execution), which makes debugging and experimentation intuitive. It is widely used in research and production for computer vision and natural language processing tasks.

2.2 Network Architecture: The MSClassifier

THE MSClassifier used in this exercise is a custom Convolutional Neural Network (CNN) designed for binary classification (MS lesion vs Control). Its architecture consists of:

- **Convolutional Layers:** Three convolutional blocks (conv1, conv2, conv3) which extract hierarchical features from the input MRI slices. Each block typically includes a convolution operation followed by batch normalization (bn1, bn2, bn3) and a ReLU activation function
- **Pooling Layers:** A flattening step converts the 2D feature maps into a 1D vector, which is passed through two fully connected (linear) layers (fc1, fc2).
- **Regularization:** Dropout is applied before the final classification layer to prevent overfitting.
- **Output:** The network outputs logits for two classes (Class 0 - Control, Class 1 - MS Lesion)

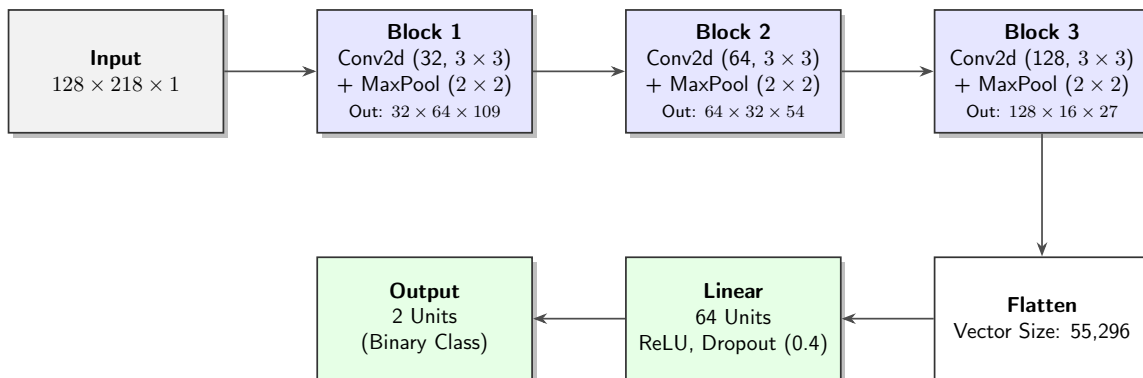


Figure 1: Architecture of the MSClassifier Model

2.3 Interpretability Methodology (XAI)

Explainable AI methods are applied to understand where the model looks when making a decision.

2.3.1 Integrated Gradients (IG)

Integrated Gradients is an axiomatic attribution method, that attributes the prediction of a deep network to its input features. [5] It works by calculating the path integral of the gradients along a straight line from a baseline input to the actual input image. It satisfies key axioms like sensitivity and implementation invariance, providing a theoretically sound explanation.

2.3.2 Gradient-weighted Class Activation Mapping (Grad-CAM)

Grad-CAM is a visualization technique that uses the gradients of a specific target class flowing into the final convolutional layer to produce a coarse localization map. [4] It highlights the regions of the image that are most important for the model's predictions. Unlike IG, which works at the pixel level, Grad-CAM provides a higher-level "heatmap" of active regions.

2.3.3 Local Interpretable Model-agnostic Explanations (LIME)

LIME explains the predictions of any classifier by approximating it locally with an interpretable model, like a linear classifier. [3] It perturbs the input and observes how the predictions change. By weighting these perturbed samples by their proximity to the original instance, LIME learns which features are locally important.

3 Framework Installation and Configuration

In this project, the **PyTorch** framework was adopted, one of the most widely used libraries for deep learning, particularly well suited for scientific research due to its flexibility, native Python integration, and advanced GPU support. The entire workflow was developed using the **Google Colab** environment, which allows code execution on accelerated hardware (GPU) without requiring complex local configurations.

The objective of this section is to provide a complete tutorial that enables the installation, configuration, and usage of the framework starting from a clean environment.

3.1 Execution Environment

All experiments were conducted on Google Colab, a cloud-based environment that provides:

- a preconfigured Linux machine;
- native support for CUDA-enabled GPUs;
- direct integration with Google Drive for data management.

The use of Colab ensures experiment reproducibility and significantly reduces setup time compared to a local installation.

3.2 Library Installation

The first step consists of installing the required libraries. PyTorch and Torchvision are used for model definition and training, while additional libraries support image processing, visualization, and interpretability techniques.

- `torch`, `torchvision`: deep learning framework;
- `opencv-python`: image reading and preprocessing;
- `numpy`: numerical operations
- `matplotlib`, `seaborn`: result visualization;
- `scikit-learn`: classification metrics
- `captum`: explainable AI techniques (Grad-CAM, Integrated Gradients);
- `lime`: LIME Image Explainer
- `segmentation-models-pytorch`: advanced utilities for CNN-based models;
- `albumentations`: data augmentation.

The installation is performed via `pip` directly within the notebook:

```
pip install torch torchvision matplotlib opencv-python captum
pip install segmentation-models-pytorch
pip install lime
```

Once the installation is completed, the libraries are imported into the notebook to verify their correct functioning.

3.3 Environment Configuration and Reproducibility

To ensure experiment reproducibility, a deterministic configuration was enforced by fixing the random seeds of PyTorch, NumPy, and Python. Additionally, non-deterministic CUDA optimizations were disabled.

```
torch.manual_seed(42)
torch.cuda.manual_seed_all(42)
np.random.seed(42)
random.seed(42)
```

This configuration is essential in academic contexts to allow for fair and reliable comparison of experimental results.

3.4 Google Drive Integration

The MSLesSeg dataset is provided as a compressed archive. To facilitate data management and preserve files across different sessions, Google Drive was used as persistent storage.

After mounting Google Drive within the Colab environment, the dataset is automatically extracted into a local directory:

```
drive.mount('/content/drive')
```

The code checks for the existence of both the archive and the destination folder, preventing redundant extractions and enabling faster startup in subsequent sessions.

3.5 Dataset Structure Verification

Once the data have been extracted, the directory structure is verified to ensure that the dataset conforms to the organization described in the official documentation. In particular, only the **axial** view is used, as required by the project guidelines.

The separation between `imagesTr` and `imagesTs` guarantees a correct patient-level split between training and test sets.

3.6 DataLoader Preparation

After verifying the dataset, custom PyTorch `Dataset` classes are used to load MRI slices into memory and provide a consistent sample interface. Each item returned by the dataset follows the triplet structure:

(image tensor, class label, lesion mask)

The lesion mask is not used as an input feature for classification. Instead, it is retained for post-hoc interpretability validation. For control samples, an empty mask is returned, ensuring a uniform interface across both classes.

The preprocessing includes a fixed resolution, skull stripping via intensity thresholding to suppress non-brain regions, and intensity normalization. During training, augmentation is applied (random flips, small rotations, brightness/contrast changes). When geometric transformations are applied, the same transform is applied to the lesion mask to maintain pixel-wise alignment between images and annotations.

3.7 Validation Split

Beyond the dataset's train/test partitioning, a validation set is created by randomly splitting the training dataset into two subsets using shuffled sample indices. Validation loss is computed at the end of each epoch and is used to drive learning-rate scheduling and to monitor generalization during training.

3.8 Data Leakage Verification

A critical aspect of the framework configuration is the prevention of *data leakage*. To this end, an automated procedure was implemented to verify that no patient appears simultaneously in both the training and test sets.

This verification is particularly important in the medical domain, where improper data splitting can lead to artificially optimistic results that do not generalize to unseen patients.

3.9 Hardware Configuration

The framework automatically detects the availability of a CUDA-enabled GPU and selects the most performant hardware available:

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

In this way, the same code can be executed seamlessly on both CPU and GPU without modifications, ensuring portability and flexibility.

4 Training

The objective of the training phase is to train a lightweight convolutional neural network on the MSLesSeg-derived 2D FLAIR axial slices, adapting the model parameters to discriminate between control (healthy) samples and MS samples.

The emphasis is not exclusively on maximizing predictive performance, but on obtaining a model whose decisions can be inspected reliably through post-hoc explainability methods (Grad-CAM, Integrated Gradients, and LIME).

4.1 Data Refinement and Preprocessing

Prior to training, the data loading strategy evolved across three iterations to improve preprocessing quality, enable interpretability validation, and make the label definition explicit. The workflow progressed from a naive cached loader (`MSLesionDatasetCached`), to a lesion-aware refinement used to study slice-level label noise (`MSLesionDatasetClean`), and finally to a consolidated implementation (`MSLesionDatasetComplete`) that standardizes outputs while preserving the original MS-vs-control classification objective.

- **Baseline loader** (`MSLesionDatasetCached`): The initial approach loaded 2D axial FLAIR slices into RAM and assigned labels purely based on folder membership (MS vs control). This is computationally efficient, but it implicitly assumes that all slices from MS subjects are equally informative for the class label.
- **Lesion-aware refinement** (`MSLesionDatasetClean`): To better understand and reduce potential slice-level label noise, an intermediate dataset version was implemented that explicitly inspects the ground-truth lesion masks (`_gt.png`) for MS samples. In this variant, MS slices were retained only when the corresponding mask contained lesion pixels. This produces a slice-level “lesion-present” positive class and reduces the risk of training on normal-appearing MS slices when the goal is lesion-driven discrimination.
- **Final consolidated dataset** (`MSLesionDatasetFinal`): The final dataset retains the efficiency of RAM caching and *does not filter* MS samples by lesion presence. Instead, it preserves the MS-vs-control label definition while returning a consistent triplet structure.

The lesion mask is *not* used as an input to the classifier; it is returned to support post-hoc interpretability validation (e.g., comparing Grad-CAM / Integrated Gradients / LIME maps against annotated lesions). For control samples, an empty (all-zero) mask is returned to preserve a uniform interface across both classes.

Preprocessing. All images are resized to a fixed spatial resolution to ensure a consistent network input. A skull-stripping step is applied using `OpenCV` intensity thresholding and bitwise masking to suppress non-brain regions, encouraging the model to focus on brain tissue rather than background artifacts.

Augmentation. During training, online data augmentation is applied (random horizontal flips, small rotations, and mild brightness/contrast changes). When geometric transforms are used, they are applied synchronously to the image and its lesion mask to maintain spatial alignment, which is essential for subsequent interpretability analysis and mask-based validation metrics.

4.2 Model configuration and training strategy

The adopted model is a convolutional neural network composed of three `Conv2D` + `Batch Normalization` + `ReLU` + `MaxPooling` blocks, followed by two fully connected layers. The inclusion of **Batch Normalization** helps stabilize gradient flow during training, while **Dropout** (with a probability of 0.4) reduces the risk of overfitting, which is particularly relevant in the medical imaging domain.

The training process was carried out using the following configuration:

- **Optimizer:** Adam;
- **Initial learning rate:** 0.001;
- **Loss function:** Cross-Entropy Loss with class weighting;
- **Batch size:** 64;

- **Number of epochs:** 20.

To compensate for a mild class imbalance, a **weighted loss function** was employed, assigning greater importance to the pathological class (lesion), thereby penalizing misclassifications of multiple sclerosis patients more heavily.

4.3 Learning rate scheduler

During training, a `ReduceLROnPlateau` scheduler was employed to dynamically adapt the learning rate based on the **validation loss**. Specifically, the learning rate is reduced by 50% whenever the validation loss does not improve for two consecutive epochs. Using validation loss rather than training loss reduces the risk of tuning the learning rate schedule toward memorization and provides a more reliable convergence signal for model selection.

4.4 Loss function behavior

Training was executed on the available compute device (GPU when `cuda` is available, otherwise CPU). Across epochs, the training loss decreases steadily, while the validation loss is monitored at the end of each epoch. The learning rate scheduler intervenes when validation loss saturates, enabling a coarse-to-fine optimisation pattern: faster learning in early epochs followed by smaller updates once the model approaches a plateau.

Because validation is derived from a random slice-level split of the training set, validation curves may appear unusually strong. For this reason, loss curves are interpreted together with interpretability-driven diagnostics (explanation map alignment with lesion masks) rather than as the sole indicator of clinically meaningful learning.

4.5 Overfitting prevention

To mitigate the risk of overfitting, several complementary strategies were adopted:

- online data augmentation (rotations, horizontal flips, brightness, and contrast variations);
- image normalization;
- use of Batch Normalization;
- Dropout in the fully connected layers;
- validation monitoring to guide learning-rate scheduling
- explicit verification of the absence of patient-level data leakage.

These design choices are particularly important in the context of medical imaging, where the ability of the model to generalize to unseen patients represents a fundamental requirement.

5 Example of Use (Inference & XAI)

This chapter details the inference pipeline and the application of XAI techniques to interpret the MSClassifier's predictions. As specified in the project requirements, the objective is not merely to classify samples but to visualize the internal reasoning of the model to verify if it correctly identifies pathological features (lesions) vs irrelevant artifacts.

5.1 Inference Methodology

The inference phase is critical for assessing the model's generalization capabilities. The procedure, implemented in the `explain_and_plot` function within the project notebook, follows a strict protocol to ensure data integrity and reproducibility:

1. Environment Setup and Model State:

- Computation is performed on the available device (`cuda` when a GPU is available, otherwise CPU).
- The model is explicitly switched to the evaluation mode through `model.eval()`. This is a crucial step that freezes the Batch Normalization running statistics (mean and variance) and disables Dropout layers, ensuring that the neural network behaves deterministically for every single forward pass.

2. Sample Selection:

- A **single 2D axial FLAIR slice** is selected from the test dataset. In the notebook this selection is random, but evaluation loaders use `shuffle=False` to guarantee deterministic ordering when required.
- **Constraint:** The selection is strictly limited to the test partition images to guarantee that sample was never seen by the network during the training phase.
- In addition to the input image, the corresponding **ground-truth lesion mask** is retrieved when available. The mask is *not* used for prediction; it is used only for post-hoc comparison and optional quantitative validation of explanation maps.

3. Tensor Preprocessing:

- The input image, originally a 2D grayscale array (128 x 218), is converted into a PyTorch tensor.
- **Dimensionality Expansion:** The system applies `unsqueeze(0)` to introduce the batch dimension, reshaping the tensor from (C, H, W) to (B, C, H, W) (specifically 1, 1, 128, 218), satisfying the input requirements of the 2D convolutional layers.

5.2 Interpretability Methodology

To mitigate the “black-box” nature of the CNN and to assess lesion-relevant reasoning, three explainability methods are applied and compared. Captum is used for gradient-based attribution, and LIME is used as a perturbation-based, model-agnostic baseline.

5.2.1 Gradient-weighted Class Activation Mapping (Grad-CAM)

Grad-CAM is applied to obtain a coarse spatial localization of the regions most influential for the predicted class.

- **Mechanism:** Grad-CAM uses gradients of the target class score with respect to the feature maps of a chosen convolutional layer to compute a class-discriminative heatmap.
- **Implementation Details:**
 - **Target layer:** Attributions are computed using the last convolutional block (`model.conv3`), which provides high-level features while retaining spatial structure.
 - **Computation:** Gradients are backpropagated from the predicted class score to the selected feature maps, and channel-wise gradient pooling is used to weight the activations.
 - **Upsampling:** The resulting heatmap is resized via bilinear interpolation to match the input resolution for visualization and comparison.

5.2.2 Integrated Gradients (IG)

Integrated Gradients provides fine-grained pixel-level attributions and is used to cross-check whether the model relies on lesion-relevant pixels rather than background regions.

- **Mechanism:** IG computes the path integral of gradients from a baseline input (e.g., an all-zero image) to the real input along a straight-line interpolation.
- **Theoretical advantage:** IG satisfies the completeness axiom, such that the sum of attributions corresponds to the difference between the model output for the input and the baseline, mitigating gradient saturation effects.
- **Context:** In this workflow, IG serves as a fine-grained verification tool complementary to Grad-CAM, particularly useful when Grad-CAM heatmaps are spatially coarse.

5.2.3 Local Interpretable Model-agnostic Explanations (LIME)

LIME is used as a model-agnostic perturbation-based explanation method and provides an additional sanity check independent of gradient computations.

- **Mechanism:** LIME perturbs the input (by masking superpixels) and observes changes in predicted probability. It then fits a simple interpretable surrogate model locally around the instance to identify influential regions.
- **Context:** Although computationally more expensive, LIME helps verify whether influential superpixels correspond to meaningful brain regions rather than background artifacts.

5.3 Visualization and Clinical Interpretation

The visualization format differs across methods to match their underlying mechanisms. In all cases, explanations are interpreted with respect to lesion plausibility, and when masks are available, qualitative alignment between saliency maps and lesion regions is assessed.

5.3.1 Grad-CAM visualization

Presented as a three-panel plot:

1. **Original image:** the grayscale MRI slice.
2. **Grad-CAM heatmap:** a normalized localization map indicating regions most influential for the predicted class.
3. **Overlay:** the heatmap superimposed on the MRI slice (e.g., $\alpha = 0.4$) to assess whether high-activation regions align with lesion locations rather than irrelevant structures.

5.3.2 Integrated Gradients visualization

Presented as a three-panel plot with post-processing:

1. **Original image:** the grayscale MRI slice.
2. **IG attribution map:** a clipped/normalized attribution map to improve interpretability by suppressing low-magnitude noise.
3. **Overlay:** the attribution map combined with the original image to display pixel-level focus.

5.3.3 LIME visualization

Presented as a two-panel plot:

1. **Original image:** the input MRI slice.
2. **Overlay:** a visualization in which the most influential superpixels are outlined or highlighted on the original image.

5.3.4 Quantitative validation against lesion masks

When lesion masks are available, explanation quality can be validated quantitatively using (i) overlap-based agreement (e.g., IoU between a thresholded explanation map and the lesion mask) and (ii) point-based agreement (pointing game), where an explanation is considered correct if the maximum attribution lies inside the lesion region. These measures complement qualitative inspection and support a more reliable interpretation of whether the model attends to lesion tissue.

6 Evaluation and Conclusion

6.1 Technical aspects: performance and reliability

From a performance perspective, PyTorch proved to be an effective framework for prototyping and training a CNN on 2D MRI FLAIR slices. Training and evaluation were stable, and the final model achieved near-perfect discriminative performance on the held-out test partition.

Figure 2 reports the confusion matrix on the test set. Out of 3853 evaluated samples, the model achieved an accuracy of 0.9982, corresponding to approximately seven misclassifications in total. The classification report indicates precision and recall values reported as 1.00 for both classes; however, these values are rounded to two decimals and therefore reflect an *almost perfect* outcome rather than a strictly error-free classifier. In practical terms, the observed error rate is extremely small for both classes, suggesting a highly separable dataset under the adopted preprocessing and training configuration.

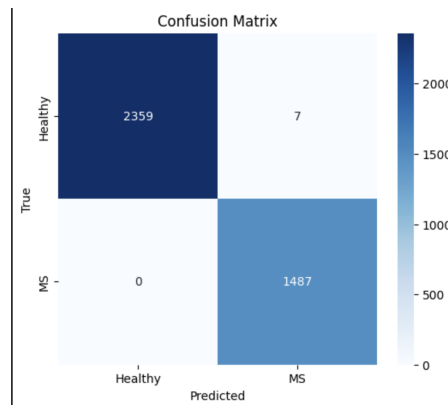


Figure 2: Confusion matrix on the test set. The model achieves near-perfect slice-level classification performance (accuracy = 0.9982 on 3853 samples).

Despite the strong predictive metrics, performance values must be interpreted in the context of the underlying label definition. In the final dataset implementation (`MSLesionDatasetFinal`), the classification objective remains *MS vs. control* at slice level, while lesion masks are used only for post-hoc evaluation of explanation maps. Therefore, high classification performance does not automatically imply that the decision mechanism is lesion-driven; this must be examined using interpretability methods and mask-based agreement metrics.

6.2 Technical aspects: interpretability and coherence of explanations

A major advantage of PyTorch is the ease of integrating interpretability techniques (e.g., Captum for gradient-based attribution and LIME for perturbation-based explanations) with full control of the computational graph. The generated visualizations provide qualitative insight into which regions influence the network’s decisions and whether these regions are compatible with clinically meaningful cues.

Figure 3 shows Grad-CAM applied to an MS example. As expected for a classification model, the activation map is often coarse and may highlight broader anatomical structures in addition to lesion-adjacent regions. Grad-CAM is therefore better interpreted as an indicator of *class-relevant evidence* rather than a lesion segmentation output. For this reason, qualitative inspection is complemented by quantitative validation against lesion masks.

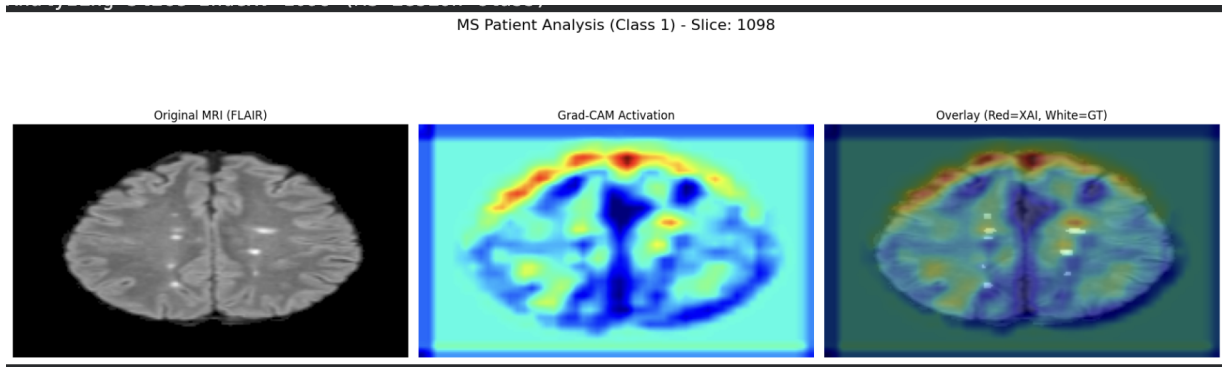


Figure 3: Grad-CAM applied to an MS (1) example. The heatmap highlights class-relevant regions but remains spatially coarse, as expected for a classification model.

To move beyond qualitative inspection, explanation maps were evaluated quantitatively against lesion masks using two criteria: (i) IoU overlap between a thresholded explanation region and the ground-truth lesion mask, and (ii) a pointing-game test assessing whether the peak attribution falls inside the lesion region (strict) or within a relaxed neighborhood around it (relaxed). These results highlight a central outcome of the project: **near-perfect classification accuracy does not necessarily translate into strong lesion-localization agreement.**

Grad-CAM (masked evaluation). Across 1487 lesion-positive slices, Grad-CAM achieved a mean IoU of 0.0197. The relaxed pointing-game hit rate was 0.0868, and the strict criterion produced 0.0034. This indicates that Grad-CAM occasionally overlaps with lesion regions, but its peak evidence rarely falls inside the lesion mask. The result is consistent with Grad-CAM’s known behaviour: it produces coarse maps that often emphasise broader anatomical patterns rather than small focal structures.

Integrated Gradients (IG). Integrated Gradients was evaluated with **SmoothGrad** and background (brain) masking to reduce attribution noise and suppress spurious background responses. Figure 4 illustrates the effect of this post-processing, yielding more stable and visually readable attribution patterns. Quantitatively, across 1487 lesion-positive slices, SmoothGrad + masking achieved a mean IoU of 0.0027, but substantially improved the pointing-game scores (relaxed hit rate 0.2495; strict hit rate 0.2468). Although IoU remains low, the higher pointing-game rates suggest that the *peak attribution* is more frequently placed within or near lesion locations, even when the overall explanation remains diffuse relative to the small lesion mask area.

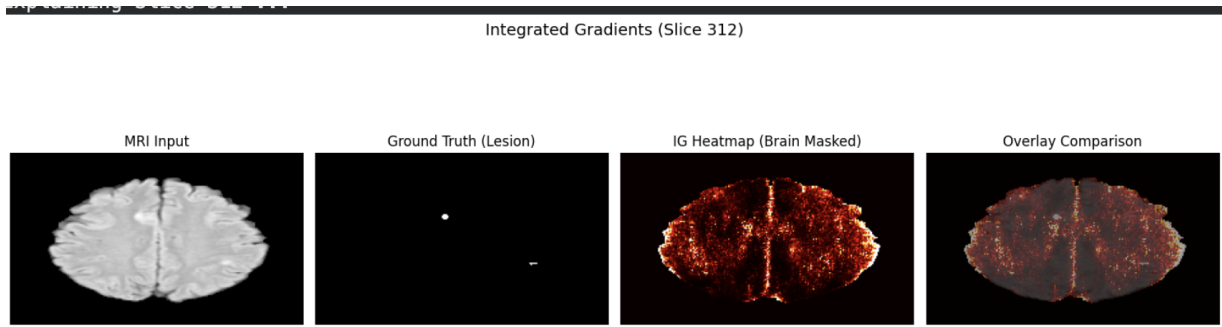


Figure 4: Integrated Gradients with SmoothGrad and background masking. Masking non-brain regions reduces noise and improves readability; smoothing stabilizes the attribution map and improves peak localisation in pointing-game tests.

LIME. LIME was evaluated on a smaller subset (20 lesion slices) due to its computational cost. In this subset, LIME achieved a mean IoU of 0.0285. The pointing-game intersection hit rate was 0.55, while relaxed and strict variants were 0.05 and 0.00 respectively. These results suggest that LIME can sometimes produce superpixel regions that intersect the lesion mask, but the peak evidence is not reliably lesion-centered. Given the small

sample size and LIME’s sensitivity to superpixel segmentation granularity and perturbation parameters, these findings are interpreted as qualitative support rather than definitive localisation performance.

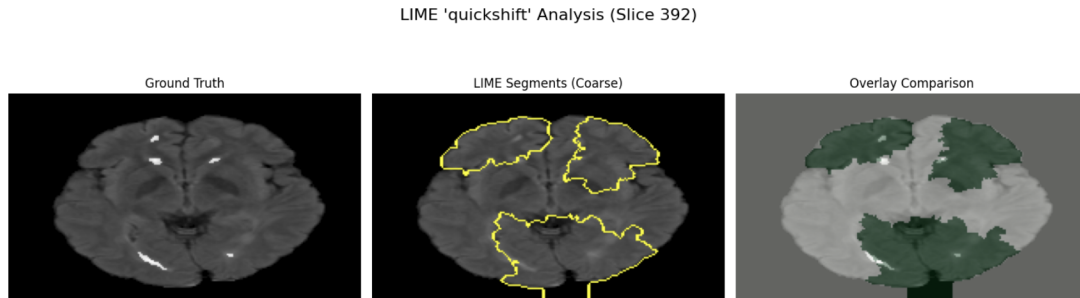


Figure 5: LIME explanation on an MS example. Highlighted superpixels represent regions that positively contribute to the MS prediction. The explanation is coarse and sensitive to superpixel segmentation, thus primarily used as qualitative support.

Overall, the interpretability analysis suggests the following: the model achieves extremely strong discrimination between MS and control slices, while the measured lesion-level agreement of explanation maps remains limited. This is not necessarily contradictory. First, the task is *slice-level classification*, whereas lesion masks represent *small focal targets*; many attribution methods are expected to be spatially diffuse and therefore penalized by IoU. Second, explanations may capture distributed cues correlated with MS (e.g., intensity patterns, tissue context, or periventricular structure) rather than sharply isolating lesion pixels. The improved pointing-game results for SmoothGrad + masked IG support the claim that, at least in a subset of cases, peak evidence is lesion-adjacent, but lesion-centric localisation is not consistently achieved.

6.3 Concluding remarks

In conclusion, PyTorch proved fully suited to the objectives of the project: it provides efficient training, robust GPU integration, and a mature interpretability ecosystem (Captum, LIME). The trained CNN demonstrates near-perfect test performance (accuracy 0.9982 on 3853 slices). However, the quantitative interpretability validation (IoU and pointing-game) indicates that, on average, explanation maps exhibit **limited overlap** with lesion masks. Therefore, the most defensible conclusion is that the classifier is effective at distinguishing MS vs. control in this dataset, but the evidence that it relies primarily on focal lesion regions is mixed.

Future improvements could strengthen reliability and clinical relevance by (i) adopting patient-grouped validation splits for model selection, (ii) adding causal perturbation tests (lesion masking vs. random masking) to quantify whether lesion pixels are truly necessary for the prediction, and (iii) aligning the learning objective more directly with lesion localisation via multi-task learning (classification + segmentation) or explicit lesion-present vs. lesion-absent slice supervision.

References

- [1] Francesco Guarnera et al. “MSLesSeg: Baseline and Benchmarking of a New Multiple Sclerosis Lesion Segmentation Dataset”. In: *Scientific Data* 12.1 (May 31, 2025), p. 920. ISSN: 2052-4463. DOI: 10.1038/s41597-025-05250-y. URL: <https://www.nature.com/articles/s41597-025-05250-y> (visited on 01/11/2026).
- [2] Adam Paszke et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. Dec. 3, 2019. DOI: 10.48550/arXiv.1912.01703. arXiv: 1912.01703 [cs]. URL: <http://arxiv.org/abs/1912.01703> (visited on 01/11/2026). Pre-published.
- [3] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. *"Why Should I Trust You?": Explaining the Predictions of Any Classifier*. Aug. 9, 2016. DOI: 10.48550/arXiv.1602.04938. arXiv: 1602.04938 [cs]. URL: <http://arxiv.org/abs/1602.04938> (visited on 01/11/2026). Pre-published.
- [4] Ramprasaath R. Selvaraju et al. “Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization”. In: *2017 IEEE International Conference on Computer Vision (ICCV)*. 2017 IEEE International Conference on Computer Vision (ICCV). Oct. 2017, pp. 618–626. DOI: 10.1109/ICCV.2017.74. URL: <https://ieeexplore.ieee.org/document/8237336> (visited on 01/11/2026).
- [5] Mukund Sundararajan, Ankur Taly, and Qiqi Yan. *Axiomatic Attribution for Deep Networks*. June 13, 2017. DOI: 10.48550/arXiv.1703.01365. arXiv: 1703.01365 [cs]. URL: <http://arxiv.org/abs/1703.01365> (visited on 01/11/2026). Pre-published.